Kurseinheit	Phase	Beschreibung	Theorie (Quelle) bzw. Praxis (Quelle)	Zeit Lehrer bzw. Schüler
		Leitidee: "Wie können wir Objekte mit einer Kamera entdecken, um damit einen ESP32 anzusteuern?"		
K1	Thema	Was sieht die Kamera?		
K2	Thema	Wie lernt eine Maschine?		
K3	Thema	KI-Modell trainieren		
K4	Thema	KI-Modell auf OpenMV Cam testen		
K5	Thema	KI-Modell verbessern		
K6	Thema	ESP32 Ansteuerung		
K7	Thema	Kreis erkennen		
K7	Theorie	Bildverarbeitung	Unter Bildverarbeitung versteht man in der Informatik und der Elektrotechnik die Verarbeitung von Signalen, die Bilder repräsentieren, beispielsweise Fotografien oder Einzelbilder aus Videos. Das Ergebnis einer Bildverarbeitung kann wiederum ein Bild sein oder auch eine Menge von Merkmalen des Eingabebildes. (wikipedia)	
K7	Theorie	Hough-Transformation	Die Hough-Transformation (Sprechweise [hʌf]) ist ein robustes globales Verfahren zur Erkennung von Geraden, Kreisen oder beliebigen anderen parametrisierbaren geometrischen Figuren in einem binären Gradientenbild, also einem Schwarz-Weiß-Bild, nach einer Kantenerkennung. (...) (wikipedia) Die Circle-Hough-Transformation (CHT) ist ein grundlegendes Verfahren zur Merkmalsextraktion in der digitalen Bildverarbeitung, das zur Erkennung von Kreisen in unvollkommenen Bildern eingesetzt wird. Die Kreiskandidaten werden durch ein „Voting“-Verfahren im Hough-Parameterraum sowie die anschließende Auswahl lokaler Maxima in einer Akkumulatormatrix ermittelt. Sie stellt eine Spezialisierung der Hough-Transformation dar. (wikipedia-en)	

K7	Theorie	Circle-Hough-Transformation	In einem zweidimensionalen Raum lässt sich ein Kreis wie folgt beschreiben: $(x - a)^2 + (y - b)^2 = r^2$ (1) wobei (a, b) der Mittelpunkt des Kreises und r der Radius ist. Ist ein Punkt (x, y) in 2D festgelegt, so lassen sich die Parameter gemäß Gleichung (1) bestimmen. Der Parameterraum ist dann dreidimensional: (a, b, r). Alle Parameter, die (x, y) erfüllen, liegen auf der Oberfläche eines umgekehrten rechtwinkligen Kegels mit der Spitze (x, y, 0). (...) (wikipedia-en)	
K7	Theorie	Bestimmung der Parameter	Bestimmung der Parameter bei **bekanntem Radius R** Ist der Radius fest vorgegeben, reduziert sich der Parameterraum auf zwei Dimensionen (die Position des Kreismittelpunkts). Jeder Punkt (x, y) auf dem ursprünglichen Kreis definiert gemäß Gleichung (1) einen Kreis mit dem Radius R und dem Mittelpunkt (x, y). Der Schnittpunkt all dieser Kreise im Parameterraum entspricht dem Mittelpunkt des ursprünglichen Kreises. Betrachte vier Punkte auf einem Kreis im Originalbild (links). Rechts ist die Kreis-Hough-Transformation dargestellt. Dabei wird vorausgesetzt, dass der Radius bekannt ist. Für jeden der vier Punkte (weiße Punkte) im Originalbild lässt sich im Hough-Parameterraum ein Kreis mit dem Mittelpunkt (x, y) und dem Radius r definieren. Zur Erfassung der Schnittpunkte wird eine Akkumulatormatrix verwendet. Im Parameterraum wird für jeden Kreis der Zählerstand (die Stimmenzahl) an denjenigen Stellen um eins erhöht, durch die der neu definierte Kreis verläuft. Anschließend lässt sich das lokale Maximum bestimmen (der rote Punkt in der Mitte der rechten Abbildung). Die Position (a, b) dieses Maximums entspricht dem Mittelpunkt des ursprünglichen Kreises. (...) Bestimmung der Kreisparameter bei **unbekanntem Radius** Da der Parameterraum dreidimensional ist, ist auch die Akkumulatormatrix dreidimensional. Man kann die **möglichen Radien durchlaufen** und für jeden Radius das zuvor beschriebene Verfahren anwenden. Schließlich werden die lokalen Maxima in der 3D-Akkumulatormatrix ermittelt. (...) (wikipedia-en)	Bis hierher: 20 Min.

K7	Aufgabe	Zielscheibe aufnehmen	Schließe die OpenMV Cam über USB an den Computer an. Starte die OpenMV IDE. Verbinde die OpenMV IDE mit der Kamera.
			Öffne Datei/Beispiele/ Feature Detection/ find_circles.py. Speichere find_circles_1.py auf dem integrierten USB-Flash-Laufwerk (Laufwerk D:).
			Ändere das Programm find_circles_1.py:
			Füge hinter Zeile 19 ein:
			`csi0.vflip(True) # Flips the image vertically`
			`csi0.hmirror(True) # Mirrors the image horizontally`
			Hinter der Zeile:
			`for c in img.find_circles(`
			Passe `threshold` an. Beginne mit `threshold=5000`
			Passe `r_max` und `r_min` an. Beginne mit `r_max=60`
			Hinter der Zeile:
			`img.draw_circle(c, color=(255, 0, 0))`
			Ergänze:
			`print("min =", c[1] - c[2])`
			`print("max =", c[1] + c[2])`
			Fokussiere nun die **Zielscheibe** mit der Kamera und beobachte das Bild im **Framebuffer-Viewer**. Bei der passenden Einstellung wird der Umriss der Zielscheibe mit einem **roten Kreis** markiert.
			Am **Seriellen Terminal** werden die **Koordinaten** des Kreis-Mittelpunkts, der **Radius** und die **Magnitude** (Konfidenzwert) angezeigt. Teste den Einfluss von Abstand und Beleuchtung der Zielscheibe. Dieses Programm läuft mit ca. 8 Frames per Second.
				10 Min.

K7	Aufgabe	RGB-Wert entnehmen	Um die Farben der Ringe zu erkennen, entnehmen wir einen **RGB-Wert** aus jedem Ring. Wir berechnen die **Koordinaten** der Mitte der Ringe mit Hilfe des Radius' und der Koordinaten des Mittelpunkts. Nur die y-Koordinate wird geändert. Die Angaben `min` und `max` am Seriellen Terminal zeigen uns, ob wir richtig liegen. Die Programmzeile: `r, g, b = img.get_pixel(c.x(), y_coord)` liefert uns das RGB-Tupel.	5 Min.

K7	Aufgabe	Farben der Ringe erkennen	Nun müssen wir noch die Farbe des Rings erkennen und die zugehörige Zahl ausgeben. Diese Arbeit übernimmt die **Funktion** `my_number()`.
			Unsere Zielscheibe enthält die Farben (Tupel):
			Gelb (255, 255, 0)
			Cyan (0, 255, 255)
			Rot (255, 0, 0)
			Schwarz (0, 0, 0)
			Grün (0, 255, 0)+D59
			Wir können nicht erwarten, dass die Kamera genau diese RGB-Tupel liefert. Deshalb müssen wir den Helligkeitswert des R - G- B-Kanals mit einem Schwellwert vergleichen:
			`r = False`
			`g = False`
			`b = False`
			`if r_brightness > r_threshold:`
			`  r = True`
			`if g_brightness > g_threshold:`
			`  g = True`
			`if b_brightness > b_threshold:`
			`  b = True`
			Zuerst testen wir den Schwellwert 127 für alle Kanäle. Später verbessern wir die Werte.
			Die Boole'schen Variablen `r, g, b` werten wir wie folgt aus:
			Gelb (True, True, False)
			Cyan (False, True, True)
			Rot (True, False, False)
			Schwarz (False, False, False)
			Grün (False, True, False)
			Die Funktion gibt dann die zugehörige **Zahl** zurück. Wenn keine Farbe erkannt wird, gibt die Funktion eine ungültige Zahl zurück.
				10 Min.